EXEMPLO 1 – TROCANDO A COR DO TEMA DA PÁGINA:

Objetivo.

Criar um Context para gerenciar o tema da aplicação com opção de trocar entre "claro" e "escuro".

Passo 1: Estrutura do projeto

Execute no terminal o seguinte comando:

```
npx create-react-app tema-context-app --template typescript
cd tema-context-app
```

Passo 2: Criar o Context

Criar a pasta `context` dentro de `src` e dentro da pasta, o arquivo `ThemeContext.tsx`

```
import { createContext, useState, ReactNode, useContext } from 'react';

type Theme = 'claro' | 'escuro';

interface ThemeContextType {
  tema: Theme;
  alternarTema: () => void;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

export const ThemeProvider = ({ children }: { children: ReactNode }) => {
  const [tema, setTema] = useState<Theme>('claro');

  const alternarTema = () => {
    setTema((prev) => (prev === 'claro' ? 'escuro' : 'claro'));
  };

  return (
    <ThemeContext.Provider value={{ tema, alternarTema }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

```
export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error('useTheme deve ser usado dentro de um ThemeProvider');
  }
  return context;
};
```

```
import { createContext, useState, ReactNode, useContext } from 'react';
```

Aqui você está importando funções e tipos do React:

- createContext → cria um contexto para compartilhar dados entre componentes sem precisar passar props manualmente em todos os níveis.
- useState → hook para criar e manipular estados dentro de componentes funcionais.
- ReactNode → tipo do TypeScript que representa qualquer coisa que pode ser renderizada pelo React (string, number, JSX, etc.).
- useContext → hook usado para consumir valores de um contexto.

```
type Theme = 'claro' | 'escuro';
```

Define um tipo literal chamado Theme, que só aceita dois valores possíveis: "claro" ou "escuro". Isso ajuda a garantir consistência no uso do tema e evita erros de digitação.

```
interface ThemeContextType {
    tema: Theme;
    alternarTema: () => void;
}
```

Define uma interface para o valor do contexto:

- tema: que será do tipo Theme (claro ou escuro).
- alternarTema: uma função sem parâmetros que troca o tema.

```
const ThemeContext = createContext<ThemeContextType | undefined>(undefined);
```

Cria o contexto com createContext. O valor inicial é undefined, porque quem vai fornecer o valor real é o ThemeProvider. O tipo usado (ThemeContextType | undefined) garante que se o hook useTheme for usado fora do provider, o código pode detectar o erro.

```
export const ThemeProvider = ({ children }: { children: ReactNode }) =>
{
    const [tema, setTema] = useState<Theme>('claro');
```

Define um componente ThemeProvider que vai envolver a aplicação e fornecer o tema atual.

- children representa os componentes filhos que vão ficar dentro do provider.
- useState inicia o tema como "claro" e fornece a função setTema para alterá-lo.

```
    const alternarTema = () => {
        setTema((prev) => (prev === 'claro' ? 'escuro' : 'claro'));
    };
```

Função responsável por alternar entre os dois temas. Ela usa a forma de setTema com callback (prev) para garantir que o valor anterior seja usado corretamente.

```
    return (
        <ThemeContext.Provider value={{ tema, alternarTema}}>
            {children}
        </ThemeContext.Provider>
    );
};
```

O ThemeProvider retorna o ThemeContext.Provider. O value passado é { tema, alternarTema }, ou seja, qualquer componente dentro do provider terá acesso ao tema atual e à função de alternância. {children} garante que os componentes filhos sejam renderizados normalmente.

```
export const useTheme = () => {
    const context = useContext(ThemeContext);
    if (!context) {
        throw new Error('useTheme deve ser usado dentro de um
ThemeProvider');
    }
    return context;
};
```

Cria um hook customizado chamado useTheme. Ele usa useContext para acessar o ThemeContext.

Se o contexto for undefined (ou seja, o hook for usado fora de um ThemeProvider), dispara um erro para ajudar no debug. Caso contrário, retorna o context, permitindo acesso ao tema e alternarTema.

Passo 3: Envolver a aplicação com o Provider

No src/index.tsx ajuste:

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';
import { ThemeProvider } from './context/ThemeContext';

const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);
root.render(
  <React.StrictMode>
    <ThemeProvider>
    <App />
    </ThemeProvider>

  </React.StrictMode>
);
```

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';
import { ThemeProvider } from './context/ThemeContext';
```

- React → biblioteca principal para construir a interface.
- ReactDOM → responsável por renderizar a aplicação React dentro do HTML.
- './index.css' → arquivo de estilos globais aplicados na aplicação.
- App → componente principal da aplicação, geralmente o ponto de entrada para os demais componentes.
- reportWebVitals → função gerada automaticamente pelo Create React App (CRA), usada para medir métricas de desempenho (opcional).
- ThemeProvider → provider que criamos anteriormente, responsável por fornecer o tema (claro ou escuro) para toda a aplicação.

```
const root = ReactDOM.createRoot(
  document.getElementById('root') as HTMLElement
);
```

Localiza o elemento no arquivo index.html. Esse elemento será o container onde a aplicação React será montada. as HTMLElement → type assertion do TypeScript, garantindo que o elemento encontrado é tratado como um elemento HTML válido. ReactDOM.createRoot cria a "raiz" do React 18 (com suporte a concurrent mode).

```
root.render(
  <React.StrictMode>
    <ThemeProvider>
      <App />
    </ThemeProvider>
  </React.StrictMode>
);
```

- root.render(...) → renderiza a aplicação dentro da div#root.
- React.StrictMode → ajuda a identificar problemas potenciais no código durante o desenvolvimento (não afeta a produção).
- ThemeProvider → envolve toda a aplicação (<App />), permitindo que qualquer componente dentro dela use o useTheme. Isso garante que App e todos os filhos tenham acesso ao estado do tema.

Passo 4: Usar o Context no componente

No arquivo `src/App.tsx`:

```
import React from 'react';
import { useTheme } from './context/ThemeContext';

function App() {
  const { tema, alternarTema } = useTheme();

  const estilo = {
    backgroundColor: tema === 'claro' ? '#fff' : '#333',
    color: tema === 'claro' ? '#000' : '#fff',
    height: '100vh',
    padding: '2rem',
  };

  return (
    <div style={estilo}>
      <h1>Tema atual: {tema}</h1>
      <button onClick={alternarTema}>Alternar Tema</button>
    </div>
  );
}

export default App;
```

```
import React from 'react';
import { useTheme } from './context/ThemeContext';
```

- Importa o React.
- Importa o hook useTheme, que criamos no ThemeContext. Ele dá acesso ao valor atual do tema (tema) e à função para alternar (alternarTema).

```
function App() {
  const { tema, alternarTema } = useTheme();
```

Declara o componente funcional App. Usa o hook useTheme para consumir o contexto. Isso retorna um objeto com duas propriedades:

- tema → string que pode ser "claro" ou "escuro".
- alternarTema → função que inverte o valor do tema.

```
  const estilo = {
    backgroundColor: tema === 'claro' ? '#fff' : '#333',
    color: tema === 'claro' ? '#000' : '#fff',
    height: '100vh',
    padding: '2rem',
  };
```

Define um objeto de estilo inline que muda conforme o tema. Se o tema for "claro" → fundo branco e texto preto. Se o tema for "escuro" → fundo cinza escuro e texto branco. O height: '100vh' faz com que a altura ocupe 100% da tela. padding: '2rem' adiciona espaço interno.

```
return (
    <div style={estilo}>
      <h1>Tema atual: {tema}</h1>
      <button onClick={alternarTema}>Alternar Tema</button>
    </div>
  );
}
```

Renderiza uma com os estilos definidos acima. Mostra um <h1> com o tema atual. Exibe um <button> que, ao ser clicado, chama alternarTema, mudando entre claro e escuro.

```
export default App;
```

Exporta o componente App como padrão, para ser usado em index.tsx.

Passo 5: Rode o projeto localmente

Execute o comando:

```
npm start
```

Referências

Documentação oficial do React Context através do link abaixo:

https://legacy.reactjs.org/docs/context.html